

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №1
з дисципліни
«ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ »
на тему :
«Розробка графічного редактора об'єктів»

Виконала:

Студентка групи ІМ-54

Дрига О.О

Варіант: 11

Перевірив:

Рекечинський Д.О.

Завдання:

1. Створити у середовищі розробки новий проект з ім'ям Lab2.
2. Скопіювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст.

Завдання згідно з варіантом (Ж = 11):

- **Масив об'єктів ($11 \bmod 3 = 2$):**

Використовується статичний масив вказівників Shape *pcshape[N]. Кількість елементів масиву $N = 11 + 100 = 111$.

- **"Гумовий" слід при вводі ($11 \bmod 4 = 3$): Пунктирна лінія чорного кольору.**

- **Увід та відображення прямокутника:**

Спосіб вводу ($11 \bmod 2 = 1$): Від центру до одного з кутів.

Відображення ($11 \bmod 5 = 1$): Чорний контур з кольоровим заповненням.

Колір заповнення ($11 \bmod 6 = 5$): Сірий.

- **Увід та відображення еліпсу:**

Спосіб вводу ($11 \bmod 2 = 1$): По двом протилежним кутам охоплюючого прямокутника.

Відображення ($11 \bmod 5 = 1$): Чорний контур з білим заповненням.

- **Позначка поточного об'єкту ($11 \bmod 2 = 1$): Відображається в заголовку вікна.**

- **Інтерфейсні та архітектурні вимоги:**

Наявність меню "Об'єкти" (з підпунктами: Крапка, Лінія, Прямокутник, Еліпс), яке розташоване між меню "Файл" та "Довідка".

У звіті обов'язково має бути схема успадкування класів (діаграма класів).

Вихідний текст головного файлу:

Lab2.cpp

```
#include <windows.h>
```

```
#include <wchar>
```

```
#include <stdio>
```

```
#include "Shapes.h"
```

```
#define IDM_POINT 1001
```

```
#define IDM_LINE 1002
```

```
#define IDM_RECT 1003
```

```
#define IDM_ELLIPSE 1004
```

```
#define IDM_EXIT 1005
```

```
#define IDM_ABOUT 1006
```

```
#define N 111
```

```
Shape* pshape[N];
```

```
int shapeCount = 0;
```

```
int currentObjectType = IDM_POINT;
```

```
bool isDrawing = false;
```

```
POINT ptStart, ptCurrent;
```

```
void UpdateWindowTitle(HWND hWnd) {
```

```
    const wchar_t* shapeName = L"Крпка";
```

```
    switch (currentObjectType) {
```

```
        case IDM_POINT: shapeName = L"Крпка"; break;
```

```
        case IDM_LINE: shapeName = L"Лінія"; break;
```

```
        case IDM_RECT: shapeName = L"Прямокутник"; break;
```

```
        case IDM_ELLIPSE: shapeName = L"Еліпс"; break;
```

```
    }
```

```
    wchar_t title[256];
```

```
    swprintf_s(title, L"Lab2 - Графічний редактор [Поточний об'єкт: %s]", shapeName);
```

```
    SetWindowTextW(hWnd, title);
```

```
}
```

```
void DrawRubberTrace(HWND hWnd, POINT p1, POINT p2) {
```

```

HDC hdc = GetDC(hWnd);

SetROP2(hdc, R2_NOTXORPEN);


HPEN hPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));

HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH));


if (currentObjectType == IDM_RECT) {

    long dx = std::abs(p2.x - p1.x);

    long dy = std::abs(p2.y - p1.y);

    Rectangle(hdc, p1.x - dx, p1.y - dy, p1.x + dx, p1.y + dy);

}

else if (currentObjectType == IDM_ELLIPSE) {

    Ellipse(hdc, min(p1.x, p2.x), min(p1.y, p2.y), max(p1.x, p2.x), max(p1.y, p2.y));

}

else {

    MoveToEx(hdc, p1.x, p1.y, NULL);

    LineTo(hdc, p2.x, p2.y);

}


SelectObject(hdc, hOldPen);

SelectObject(hdc, hOldBrush);

DeleteObject(hPen);

ReleaseDC(hWnd, hdc);

}

HMENU CreateAppMenu() {

```

```

HMENU hMenu = CreateMenu();

HMENU hFileMenu = CreatePopupMenu();

HMENU hObjectsMenu = CreatePopupMenu();

HMENU hHelpMenu = CreatePopupMenu();


AppendMenuW(hFileMenu, MF_STRING, IDM_EXIT, L"Вихід");


AppendMenuW(hObjectsMenu, MF_STRING, IDM_POINT, L"Крапка");

AppendMenuW(hObjectsMenu, MF_STRING, IDM_LINE, L"Лінія");

AppendMenuW(hObjectsMenu, MF_STRING, IDM_RECT, L"Прямокутник");

AppendMenuW(hObjectsMenu, MF_STRING, IDM_ELLIPSE, L"Еліпс");


AppendMenuW(hHelpMenu, MF_STRING, IDM_ABOUT, L"Про програму");


AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");

AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hObjectsMenu, L"Об'єкти");

AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Довідка");


return hMenu;

}


LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {

    switch (message) {

    case WM_CREATE:

        SetMenu(hWnd, CreateAppMenu());

        UpdateWindowTitle(hWnd);

        break;

```

```
case WM_COMMAND: {

    int wmId = LOWORD(wParam);

    switch (wmId) {

        case IDM_POINT:

        case IDM_LINE:

        case IDM_RECT:

        case IDM_ELLIPSE:

            currentObjectType = wmId;

            UpdateWindowTitle(hWnd);

            break;

        case IDM_ABOUT:

            MessageBoxW(hWnd, L"Лабораторна робота №2\nВаріант №11\nВиконала: студентка Дрига Олександра  
ІМ-54", L"Про програму", MB_OK | MB_ICONINFORMATION);

            break;

        case IDM_EXIT:

            DestroyWindow(hWnd);

            break;

    }

    break;

}

case WM_LBUTTONDOWN:

    ptStart.x = LOWORD(lParam);

    ptStart.y = HIWORD(lParam);

    ptCurrent = ptStart;

    isDrawing = true;
```

```

if (currentObjectType == IDM_POINT) {

    if (shapeCount < N) {

        pcshape[shapeCount] = new PointShape();

        pcshape[shapeCount]->Set(ptStart.x, ptStart.y, ptStart.x, ptStart.y);

        shapeCount++;

        InvalidateRect(hWnd, NULL, TRUE);

    }

    isDrawing = false;

}

else {

    DrawRubberTrace(hWnd, ptStart, ptCurrent);

}

break;

case WM_MOUSEMOVE:

    if (isDrawing) {

        DrawRubberTrace(hWnd, ptStart, ptCurrent);

        ptCurrent.x = LOWORD(lParam);

        ptCurrent.y = HIWORD(lParam);

        DrawRubberTrace(hWnd, ptStart, ptCurrent);

    }

    break;

case WM_LBUTTONDOWN:

    if (isDrawing) {

        DrawRubberTrace(hWnd, ptStart, ptCurrent);

```

```
isDrawing = false;
```

```
ptCurrent.x = LOWORD(lParam);
```

```
ptCurrent.y = HIWORD(lParam);
```

```
if (shapeCount < N) {
```

```
    switch (currentObjectType) {
```

```
        case IDM_LINE:
```

```
            pcshape[shapeCount] = new LineShape();
```

```
            break;
```

```
        case IDM_RECT:
```

```
            pcshape[shapeCount] = new RectangleShape();
```

```
            break;
```

```
        case IDM_ELLIPSE:
```

```
            pcshape[shapeCount] = new EllipseShape();
```

```
            break;
```

```
    }
```

```
    pcshape[shapeCount]->Set(ptStart.x, ptStart.y, ptCurrent.x, ptCurrent.y);
```

```
    shapeCount++;
```

```
    InvalidateRect(hWnd, NULL, TRUE);
```

```
}
```

```
}
```

```
break;
```

```
case WM_PAINT: {
```

```
    PAINTSTRUCT ps;
```

```
    HDC hdc = BeginPaint(hWnd, &ps);
```

```

    for (int i = 0; i < shapeCount; i++) {

        if (pcshape[i] != nullptr) {

            pcshape[i]->Show(hdc);

        }

    }

    EndPaint(hWnd, &ps);

    break;

}

case WM_DESTROY:

    for (int i = 0; i < shapeCount; i++) {

        delete pcshape[i];

    }

    PostQuitMessage(0);

    break;

default:

    return DefWindowProcW(hWnd, message, wParam, lParam);

}

return 0;

}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {

    const wchar_t szClassName[] = L"Lab2WindowClass";

```

```

WNDCLASSEXW wc = { 0 };

wc.cbSize = sizeof(WNDCLASSEXW);

wc.style = CS_HREDRAW | CS_VREDRAW;

wc.lpfnWndProc = WndProc;

wc.hInstance = hInstance;

wc.hCursor = LoadCursor(NULL, IDC_ARROW);

wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

wc.lpszClassName = szClassName;


if (!RegisterClassExW(&wc)) return 0;


HWND hWnd = CreateWindowExW(

    0, szClassName, L"Lab2", WS_OVERLAPPEDWINDOW,

    CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,

    NULL, NULL, hInstance, NULL

);


if (!hWnd) return 0;


ShowWindow(hWnd, nCmdShow);

UpdateWindow(hWnd);


MSG msg;

while (GetMessage(&msg, NULL, 0, 0)) {

    TranslateMessage(&msg);

    DispatchMessage(&msg);

}

```

```
    return (int)msg.wParam;

}
```

shape.h

```
#ifndef SHAPE_H

#define SHAPE_H

#include <windows.h>

class Shape {

protected:

    long xs1, ys1, xs2, ys2;

public:

    Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}

    virtual ~Shape() {}

    virtual void Set(long x1, long y1, long x2, long y2) {

        xs1 = x1; ys1 = y1;

        xs2 = x2; ys2 = y2;

    }

    virtual void Show(HDC hdc) = 0;

};

#endif
```

Shapes.h

```
#ifndef SHAPES_H
```

```
#define SHAPES_H
```

```
#include "Shape.h"
```

```
#include <cmath>
```

```
class PointShape : public Shape {
```

```
public:
```

```
void Show(HDC hdc) override {
```

```
    SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));
```

```
    MoveToEx(hdc, xs1 - 1, ys1, NULL); LineTo(hdc, xs1 + 2, ys1);
```

```
    MoveToEx(hdc, xs1, ys1 - 1, NULL); LineTo(hdc, xs1, ys1 + 2);
```

```
}
```

```
};
```

```
class LineShape : public Shape {
```

```
public:
```

```
void Show(HDC hdc) override {
```

```
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
```

```
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
```

```
    MoveToEx(hdc, xs1, ys1, NULL);
```

```
    LineTo(hdc, xs2, ys2);
```

```
    SelectObject(hdc, hOldPen);
```

```
    DeleteObject(hPen);
```

```
}
```

```
};
```

```
class RectangleShape : public Shape {
```

```
public:
```

```
void Show(HDC hdc) override {
```

```
    long dx = std::abs(xs2 - xs1);
```

```
    long dy = std::abs(ys2 - ys1);
```

```
    long left = xs1 - dx;
```

```
    long top = ys1 - dy;
```

```
    long right = xs1 + dx;
```

```
    long bottom = ys1 + dy;
```

```
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
```

```
    HBRUSH hBrush = CreateSolidBrush(RGB(128, 128, 128));
```

```
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
```

```
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);
```

```
    Rectangle(hdc, left, top, right, bottom);
```

```
    SelectObject(hdc, hOldPen);
```

```
    SelectObject(hdc, hOldBrush);
```

```
    DeleteObject(hPen);
```

```
    DeleteObject(hBrush);
```

```
}
```

```
};
```

```

class EllipseShape : public Shape {

public:

    void Show(HDC hdc) override {

        long left = min(xs1, xs2);

        long top = min(ys1, ys2);

        long right = max(xs1, xs2);

        long bottom = max(ys1, ys2);


        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hBrush = (HBRUSH)GetStockObject(WHITE_BRUSH);


        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);


        Ellipse(hdc, left, top, right, bottom);


        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

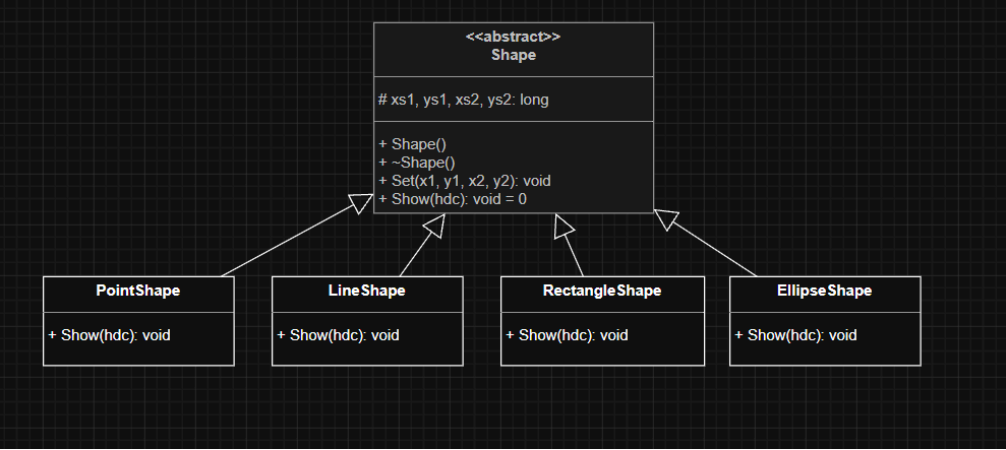
    }

};

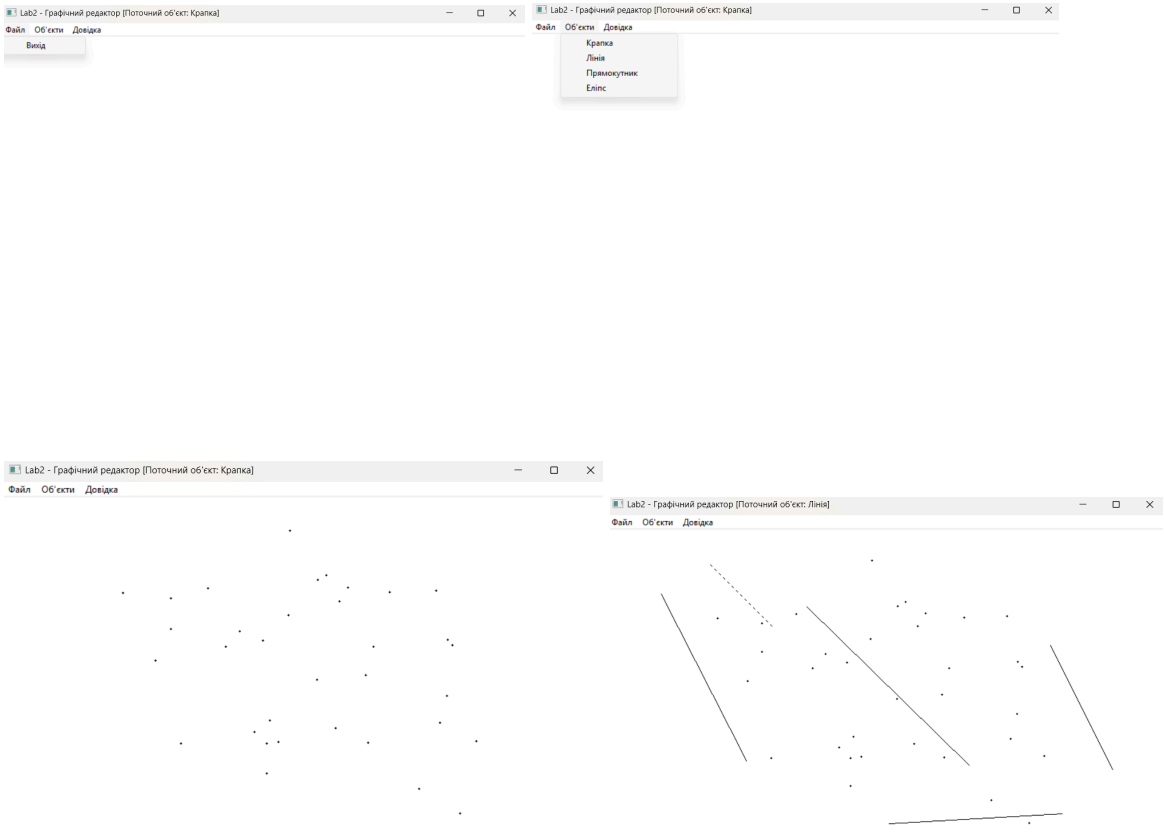
#endif

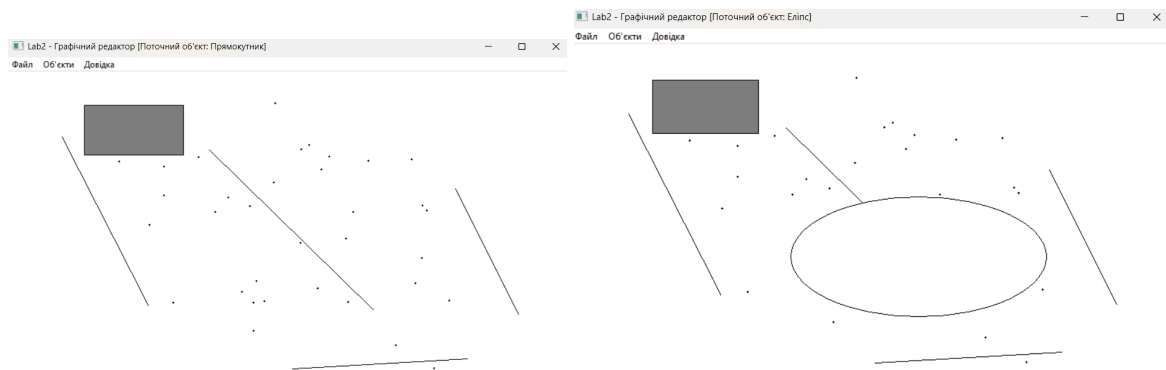
```

Схеми, діаграми згідно завданню:



Ілюстрації:





Висновок:

у ході виконання лабораторної роботи було засвоєно базові принципи об'єктно-орієнтованого програмування, зокрема абстракцію, успадкування та поліморфізм, на прикладі розробки графічного редактора мовою C++ з використанням WinAPI. Було побудовано ієрархію класів на основі абстрактного базового класу Shape та чотирьох дочірніх класів PointShape, LineShape, RectangleShape і EllipseShape. Програму повністю реалізовано відповідно до вимог варіанта №11: налаштовано інтерактивне малювання за допомогою «гумового» сліду пунктирною лінією, побудову прямокутника від центру до кута, відповідні кольори контурів і заповнень, головне меню програми та відображення типу поточного об'єкта в заголовку вікна. Також побудовано та перевірено UML-діаграму класів, яка підтверджує модульність і коректність архітектури розробленої програми.